NGHIÊN CỨU PHƯƠNG PHÁP ĐỒNG BỘ HÓA VÀ HỢP NHẤT ĐA PHƯƠNG THỨC CHO NHẬN DẠNG TIẾNG NÓI KẾT HỢP ÂM THANH VÀ HÌNH ẢNH (AVSR) CHO NGÔN NGỮ ÍT TÀI NGUYÊN: TIẾNG VIỆT

ĐỀ CƯƠNG NGHIÊN CỨU & THIẾT KẾ HỆ THỐNG

Lĩnh vực: Xử lý ngôn ngữ tự nhiên & Thị giác máy tính
Hướng nghiên cứu: Audio-Visual Speech Recognition (AVSR)
Ngôn ngữ mục tiêu: Tiếng Việt
Thời gian: 2025 – 2026

Thiết kế kiến trúc MOTA-v2 với ba cải tiến cốt lõi:

(1) Router-gated Modality Fusion,
(2) Synchronization-aware Preprocessing,
(3) Noise-robust Quality Estimation.
Kiến trúc Decoder theo lộ trình 3 cấp (Level 1–3), phù hợp với low-resource setting.

Pain point cốt lõi: Fusion model hoạt động trên benchmark nhưng thất bại khi audio nhiễu hoặc video che miệng / blur / pose lệch. MediaPipe là bottleneck chính ở pipeline cũ.

Mục lục

PHẦN I: ĐẶT VẤN ĐỀ VÀ BÀI TOÁN **1**

1 Giới thiệu đề tài **2**
1.1 Tổng quan 2
1.2 Tính cấp thiết 2
1.3 Mục tiêu nghiên cứu 3

2 Tổng quan tài liệu và nghiên cứu liên quan **4**
2.1 Evolution của AVSR (2022–2026) 4
2.2 Các phương pháp Fusion chính 4
2.2.1 Element-wise Gating (QualityGate) – Baseline MOTA 4
2.2.2 Optimal Transport Fusion (MQOT) – SOTA 4
2.3 Pain Points trong codebase MOTA 5
2.3.1 Pain Point 1: Fusion Breakdown 5
2.3.2 Pain Point 2: MQOT GuidedAttention Bug (P0-3) 5
2.3.3 Pain Point 3: MediaPipe là Bottleneck Pipeline 5

PHẦN II: PHƯƠNG PHÁP ĐỀ XUẤT **6**

3 Kiến trúc MOTA-v2 **7**
3.1 Tổng quan kiến trúc ở mức Fusion 7
3.2 Stage 0.5: SyncPreprocessor – Đồng bộ hóa Feature-level 7
3.2.1 Vấn đề 7
3.2.2 Lag Detection bằng Cross-Correlation 8
3.2.3 Cubic Interpolation Alignment 8
3.3 Stage 1: Router-Gated Fusion (RGF) – Thay thế QualityGate 8
3.3.1 Motivation: Tại sao cần chunk-level routing thay vì frame-level gating? 8
3.3.2 Kiến trúc chi tiết từng bước 9
3.3.3 Tại sao RGF tốt hơn QualityGate? 10
3.4 Stage 1+: Noise-Robust Quality Estimator (NRQE) 10
3.4.1 Motivation 10
3.4.2 Uncertainty-aware Quality Score 10
3.5 Stage 2: MQOT-v2 – Unbalanced Sinkhorn và Fixed GuidedAttention . . . 11
3.5.1 Unbalanced Sinkhorn: Cho phép $T_a \neq T_v$ 11
3.5.2 Learnable Epsilon với Annealing 11
3.5.3 Fixed GuidedAttention: Tích hợp Transport Plan vào Attention . . 12

4 Lộ trình kiến trúc Decoder: Từ Level 1 đến Level 3 **13**
4.1 Tại sao không dùng LLM Decoder ngay từ đầu? . . . 13
4.2 Level 1: Baseline Decoder (Month 1–2) . . . 14
4.3 Level 2: Enhanced Decoder (Month 3–4) . . . 14
4.3.1 Option A: CTC + Bi-LSTM Language Model . . . 14
4.3.2 Option B: CTC + LoRA Adapters (HLoRA-style) . . . 14
4.4 Level 3: Advanced Decoder (Month 5–6+) . . . 15
4.4.1 Option A: OTTC – OT-based CTC thay CTC Loss . . . 15
4.4.2 Option B: CTC + FlexCTC Beam Search . . . 15
4.5 Tổng hợp lộ trình Decoder . . . 15

5 Hệ thống và Pipeline huấn luyện **17**
5.1 Tổng quan Pipeline . . . 17
5.2 Loss Function . . . 17
5.3 Huấn luyện: Curriculum và Stability . . . 18

PHẦN III: THỰC NGHIỆM VÀ KẾT QUẢ **19**

6 Thiết kế thực nghiệm **20**
6.1 Dataset . . . 20
6.2 Baselines so sánh . . . 20
6.3 Metrics . . . 20
6.4 Training Config . . . 21

7 Kết quả dự kiến **22**
7.1 Kết quả dự kiến trên ViCocktail . . . 22
7.2 Ablation Studies . . . 23
7.3 Robustness Analysis . . . 23

PHẦN IV: TRIỂN KHAI VÀ GANT CHART **24**

8 Lộ trình triển khai 6 tháng **25**
8.1 Gant Chart . . . 25
8.2 Chi tiết từng Phase . . . 25
8.3 Risk Management . . . 26

PHẦN V: KẾT LUẬN VÀ ĐỊNH HƯỚNG **27**

9 Kết luận **28**
9.1 Tóm tắt đóng góp . . . 28
9.2 Định hướng tương lai . . . 28

Tài liệu tham khảo **30**

A Các Bug P0 cần fix trước khi training **31**

B File Structure dự kiến **32**

PHẦN I: ĐẶT VẤN ĐỀ VÀ BÀI TOÁN

Chương 1

Giới thiệu đề tài

1.1 Tổng quan

AVSR (Audio-Visual Speech Recognition) là bài toán nhận dạng tiếng nói sử dụng đồng thời hai luồng dữ liệu: **đặc trưng âm thanh** từ Whisper encoder ($[B, T_a, 768]$) và **đặc trưng hình ảnh** từ ResNet18 ($[B, T_v, 512]$). Trên benchmark lý tưởng, WER dưới 5% [13]. Tuy nhiên, trên ngôn ngữ ít tài nguyên như tiếng Việt, hiệu suất thực tế còn rất xa so với tiếng Anh [11].

Bài toán cốt lõi đặt ra: *làm thế nào để fusion model tổng hợp thông tin audio và visual một cách đáng tin cậy khi một trong hai modality bị suy giảm nghiêm trọng về chất lượng?* Đây là pain point chưa được giải quyết triệt để trong các nghiên cứu hiện tại.

1.2 Tính cấp thiết

1. **Pain point của mô hình (model-level):** Fusion model hoạt động trên benchmark nhưng thất bại khi:

   - Audio bị nhiễu nền (background noise, reverberation)
   - Video bị che miệng (hand occlusion), motion blur, pose lệch
   - Temporal misalignment giữa audio và visual frames

   Vấn đề không chỉ ở fusion mà còn ở **ước lượng độ tin cậy từng modality theo thời gian**.

2. **Bottleneck của preprocessing (pipeline-level):** Trong pipeline cũ, MediaPipe FaceMesh là chỗ thắt cổ chai:

   - Chạy CPU-only để tránh EGL context conflict với PyTorch CUDA, tốc độ chậm ($\sim$30 phút cho 1 giờ video)
   - Face detection thất bại với pose lệch ($> 30°$ yaw)
   - Temporal landmark tracking không ổn định
   - Không cung cấp quality signal cho downstream fusion model

3. **Khoảng trống research cho tiếng Việt:** Tiếng Việt có 6 thanh điệu; nhiều phoneme chỉ phân biệt được bởi hình ảnh môi (ví dụ: *bà / bá / bả / bạ*). Nghiên cứu AVSR cho tiếng Việt còn rất hạn chế.

1.3 Mục tiêu nghiên cứu

Thiết kế kiến trúc MOTA-v2 với ba cải tiến cốt lõi ở mức fusion:

1. **Router-Gated Fusion (RGF):** Thay element-wise gating bằng learned chunk-level routing, model tự học khi nào dùng audio, khi nào dùng visual.
2. **SyncPreprocessor:** Feature-level synchronization correction không phụ thuộc MediaPipe, giảm temporal misalignment.
3. **Noise-Robust Quality Estimator (NRQE):** Ước lượng độ tin cậy từng modality với uncertainty-aware scoring.
4. **Kiến trúc Decoder theo lộ trình 3 cấp (Level 1–3):** Phù hợp low-resource setting, từ CTC+N-gram LM (Level 1) đến CTC+LoRA (Level 2), đến advanced OT fusion (Level 3). Không dùng full LLM decoder trong giai đoạn này.
5. Thực nghiệm trên dataset GRID + ViCocktail, so sánh WER với baseline MOTA.

Chương 2

Tổng quan tài liệu và nghiên cứu liên quan

2.1 Evolution của AVSR (2022–2026)

AVSR tiến hóa qua nhiều giai đoạn: AV-HuBERT (2022) [13] đặt nền tảng với masked multimodal cluster prediction; Auto-AVSR (2023) [10] tự động hóa tạo labels; Whisper-Flamingo (2024) [12] dùng Whisper encoder kết hợp Flamingo-style fusion; AlignMamba (2025) [8] dùng OT-based alignment với Mamba backbone; OTTC (2025) [4] thay CTC bằng OT loss; AVUR-LLM (2025) [2] và Llama-AVSR (2025) [3] dùng LLM decoder; Omni-AVSR (2025) [9] thống nhất ASR/VSR/AVSR trong một framework.

2.2 Các phương pháp Fusion chính

2.2.1 Element-wise Gating (QualityGate) – Baseline MOTA

MOTA hiện tại dùng QualityGate để fuse audio và visual:

$$\mathbf{f}_t = w_t^{(a)}\mathbf{a}_t + w_t^{(v)}\mathbf{v}_t, \qquad [w_t^{(a)}, w_t^{(v)}]^\top = \mathrm{Softmax}\Big(\mathrm{MLP}\left([\mathbf{a}_t; \mathbf{v}_t; q_t^{(a)}; q_t^{(v)}]\right)\Big) \tag{2.1}$$

Hạn chế cốt lõi: (i) Linear interpolation cho temporal alignment giả định uniform motion; (ii) Quality scoring độc lập, không xét cross-modal correlation; (iii) Element-wise fusion không model được temporal dependencies.

Chi tiết: $\mathbf{a}_t \in \mathbb{R}^{256}$ và $\mathbf{v}_t \in \mathbb{R}^{256}$ được concatenate cùng hai quality scalars thành vector 514chiều. MLP biến đổi vector 514chiều thành 2 scalars, sau đó Softmax cho $w_t^{(a)} + w_t^{(v)} = 1$. Cách này không xét rằng audio frame t nên align với visual frame nào – nó giả định proportional alignment: audio frame i align với visual frame $\lfloor i \cdot T_v/T_a \rfloor$. Thực tế speech rate không đều (nhanh / chậm / pause), nên giả định này gây systematic misalignment.

2.2.2 Optimal Transport Fusion (MQOT) – SOTA

MQOT dùng Sinkhorn để tìm transport plan $P \in \mathbb{R}^{T_a \times T_v}$:

$$P^* = \mathrm{argmin}_{P \in \mathcal{U}(a,b)} \langle P, C \rangle + \varepsilon\, \mathrm{KL}\big(P \parallel \mathbf{a} \otimes \mathbf{b}\big) \tag{2.2}$$

với $\mathcal{U}(a,b) = \{P \in \mathbb{R}_+^{T_a \times T_v} \mid P\mathbf{1}_{T_v} = \mathbf{a},\ P^\top \mathbf{1}_{T_a} = \mathbf{b}\}$ là doubly stochastic matrices với marginals $\mathbf{a}, \mathbf{b}$; $C_{ij} = C_{ij}^{\text{feat}} + \lambda_t C_{ij}^{\text{time}} + \lambda_q C_{ij}^{\text{qual}}$ là tổng của feature similarity cost, temporal smoothness cost, và quality penalty.

Sinkhorn giải phương trình (2.2) bằng iterative updates:

$$\mathbf{u}^{(k+1)} = -\varepsilon \odot \text{logsumexp}_{\text{col}}\left(-\frac{C}{\varepsilon} + \mathbf{v}^{(k)} \otimes \mathbf{1}\right), \qquad \mathbf{v}^{(k+1)} = -\varepsilon \odot \text{logsumexp}_{\text{row}}\left(-\frac{C^\top}{\varepsilon} + \mathbf{u}^{(k+1)} \otimes \mathbf{1}\right) \tag{2.3}$$

Hạn chế trong MOTA gốc: (1) GuidedAttention bug: transport plan được truyền vào `nn.MultiheadAttention.attn_mask` với shape $[B \times H, T_a, T_v]$ nhưng PyTorch mong đợi $[L, S]$ (2D); (2) Balanced OT giả định $T_a = T_v$; (3) epsilon cố định không annealing.

2.3 Pain Points trong codebase MOTA

2.3.1 Pain Point 1: Fusion Breakdown

QualityGate (Eq. 2.1) có hai hạn chế nghiêm trọng:

(P1a) Linear interpolation gây misalignment. Code QualityGate biến đổi $\mathbf{v} \in \mathbb{R}^{T_v \times 256}$ thành $\tilde{\mathbf{v}} \in \mathbb{R}^{T_a \times 256}$ bằng `F.interpolate(mode='linear')`. Phép biến đổi này giả định tốc độ nói đều: audio frame i tương ứng với visual frame $\lfloor i \cdot T_v / T_a \rfloor$. Trên thực tế, người nói có thể nói nhanh ở đầu câu và chậm ở cuối, hoặc có khoảng nghỉ (pause) dài. Khi đó frame i thực tế nên align với visual frame khác rất xa so với công thức trên.

(P1b) Independent quality scoring. Hai MLP riêng biệt tính $q_t^{(a)}$ và $q_t^{(v)}$ hoàn toàn độc lập. Nếu cả audio lẫn visual đều có chất lượng thấp tại frame t, model vẫn cố gắng fuse hai modality bị degraded, dẫn đến **double degradation**. Không có cơ chế để model phát hiện rằng "cả hai modality đều không đáng tin cậy tại frame này, nên bỏ qua frame này"hoặc "chỉ dùng modality nào đáng tin hơn".

2.3.2 Pain Point 2: MQOT GuidedAttention Bug (P0-3)

Phương trình (3.13) mô tả đúng cách tích hợp transport plan vào attention. Code gốc cố gắng truyền attn_mask với shape $[B \times H, T_a, T_v]$ vào `nn.MultiheadAttention`. PyTorch broadcast mask này theo cách sai, dẫn đến transport plan không ảnh hưởng đến attention scores.

2.3.3 Pain Point 3: MediaPipe là Bottleneck Pipeline

Pipeline cũ có dependency tuyến tính: MP FaceMesh → Landmarks → Mouth Crop → ResNet18 → Features → MOTA. Mỗi stage thất bại đều propagate error lên model mà model không biết. MediaPipe xử lý frame-by-frame, không có temporal consistency, và không cung cấp quality signal cho downstream fusion.

PHẦN II: PHƯƠNG PHÁP ĐỀ XUẤT

Chương 3

Kiến trúc MOTA-v2

3.1 Tổng quan kiến trúc ở mức Fusion

MOTA-v2 giữ nguyên 6-stage pipeline từ MOTA gốc, nhưng thay thế Stage 1 (Quality-Gate) bằng hai module mới, và nâng cấp Stage 2 (MQOT) thành MQOT-v2:

1. **Stage 0.5 – SyncPreprocessor:** Cross-correlation lag detection và alignment correction trước khi đưa vào model.
2. **Stage 1 – Router-Gated Fusion (RGF):** Chunk-level routing với 3 modes: Audio-dominant, Visual-dominant, Fusion. Tích hợp NRQE để uncertainty-aware routing.
3. **Stage 2 – MQOT-v2:** Unbalanced Sinkhorn cho phép $T_a \neq T_v$, learnable epsilon với annealing, và fixed GuidedAttention.

Data flow đầy đủ:

$$\underbrace{[B, T_a, 768]}_{\text{Audio}} \rightarrow \underbrace{\text{SyncPreprocessor}}_{\text{Stage 0.5}} \rightarrow \underbrace{\text{AudioProj } [B, T_a, 256]}_{\text{Stage 1a}}$$

$$\underbrace{[B, T_v, 512]}_{\text{Visual}} \rightarrow \underbrace{\text{VisualProj } [B, T_v, 256]}_{\text{Stage 1a}}$$

$$\downarrow$$

$$\underbrace{\text{RGF (RGF + NRQE)}}_{\text{Stage 1b}} \rightarrow \underbrace{\text{MQOT-v2 + GuidedAttentionV2}}_{\text{Stage 2}} \rightarrow \underbrace{4 \times \text{ConformerBlock}}_{\text{Stage 3}} \rightarrow \underbrace{\text{HybridDecoder}}_{\text{Stage 4 (Leve}}$$

3.2 Stage 0.5: SyncPreprocessor – Đồng bộ hóa Feature-level

3.2.1 Vấn đề

$T_a \neq T_v$ do frame-rate mismatch và **audio-visual asynchrony** tự nhiên. Lag audio-video thường 50–200 ms trong video thực tế. Nếu không correct lag này, alignment giữa audio và visual frame sẽ bị systematic offset ngay từ đầu.

3.2.2 Lag Detection bằng Cross-Correlation

Ý tưởng: tìm shift τ sao cho audio và visual features có correlation cao nhất:

$$\hat{\tau} = \text{argmax}_{\tau \in [-\tau_{\max}, \tau_{\max}]} \sum_t \mathbf{a}_t^\top \mathbf{v}_{t+\tau}, \qquad \tau_{\max} = 10 \text{ frames} \tag{3.1}$$

Tại sao dùng common space? Vì cosine similarity $\mathbf{a}_t^\top \mathbf{v}_{t+\tau}$ chỉ meaningful khi $\mathbf{a}$ và $\mathbf{v}$ cùng một không gian biểu diễn. Công thức (3.1) tương đương với convolution: đặt $\tilde{\mathbf{v}}$ là $\mathbf{v}$ đảo ngược theo thời gian, rồi convolution $\mathbf{a} * \tilde{\mathbf{v}}$ tại position τ chính là correlation sum. Peak của convolution cho biết optimal lag.

Độ phức tạp: $O(B \cdot T_a \cdot \tau_{\max})$. Với $\tau_{\max} = 10$ và $T_a \sim 300$, đây là overhead nhỏ.

3.2.3 Cubic Interpolation Alignment

Sau khi detect $\hat{\tau}$, align visual bằng cubic interpolation:

$$\tilde{\mathbf{v}}_t = \text{CubicSpline}\left(\mathbf{v}_{t+\hat{\tau}}^{\text{align}}\right) \tag{3.2}$$

Tại sao cubic thay vì linear? Cubic spline giữ được smoothness của feature trajectory – derivative liên tục – trong khi linear interpolation tạo ra "góc cạnh"ở mỗi data point. Với speech features có temporal smoothness tự nhiên, cubic interpolation giảm aliasing effect khi resampling.

SyncPreprocessor cũng trả về lag confidence:

$$c_{\text{lag}} = \frac{\max_\tau \sum_t \mathbf{a}_t^\top \mathbf{v}_{t+\tau}}{\sum_t \mathbf{a}_t^\top \mathbf{a}_t} \in [0, 1] \tag{3.3}$$

Nếu c_{lag} thấp (e.g., < 0.3), audio và visual thực sự không correlated – signal này được truyền xuống NRQE để giảm trọng số fusion.

3.3 Stage 1: Router-Gated Fusion (RGF) – Thay thế QualityGate

3.3.1 Motivation: Tại sao cần chunk-level routing thay vì frame-level gating?

QualityGate (Eq. 2.1) quyết định fusion weight tại mỗi frame độc lập, không có **context xung quanh**. Nếu frame t có $w_t^{(a)} = 0.7$, model không biết rằng frames $t-1, t, t+1$ cũng nên ưu tiên audio.

RGF khắc phục bằng:

1. **Chunk-level:** Gom 15 frames liên tiếp thành một chunk. Router quyết định tại chunk level thay vì frame level.
2. **Hard routing:** Chọn đúng một trong 3 modes, thay vì weighted sum liên tục.
3. **Context-aware:** Chunk encoder nhìn toàn bộ 15 frames, có đủ context để quyết định.

3.3.2 Kiến trúc chi tiết từng bước

Bước 1: Chunk Encoding. Input $\mathbf{a} \in \mathbb{R}^{T_a \times D}$, $\mathbf{v} \in \mathbb{R}^{T_v \times D}$. Gom thành $C = \lfloor T_a/K \rfloor$ chunks, mỗi chunk $K = 15$ frames:

$$\mathbf{a}^{(c)} \in \mathbb{R}^{K \times D}, \quad \mathbf{v}^{(c)} \in \mathbb{R}^{K \times D}, \quad c = 1, \ldots, C$$

Sau đó flatten và project:

$$\mathbf{a}_c^{\text{ctx}} = \text{ReLU}\big(\mathbf{W}_{\text{ctx}} \cdot \text{flatten}(\mathbf{a}^{(c)}) + \mathbf{b}_{\text{ctx}}\big) \in \mathbb{R}^D$$

Bottleneck architecture: $K \cdot D = 15 \times 256 = 3840$ dimensions $\to$ 512 dimensions. Bottleneck buộc encoder phải học compressed representation chứa essence của chunk.

Bước 2: Router Decision. Router nhận $\mathbf{a}_c^{\text{ctx}}$ và $\mathbf{v}_c^{\text{ctx}}$ rồi tính routing logits:

$$\boldsymbol{\pi}_c = \text{MLP}_{\text{router}}\Big(\big[\mathbf{a}_c^{\text{ctx}};\ \mathbf{v}_c^{\text{ctx}};\ \bar{q}_c^{(a)};\ \bar{q}_c^{(v)}\big]\Big) \in \mathbb{R}^3 \tag{3.4}$$

Sau đó Gumbel-Softmax:

$$\hat{\boldsymbol{\pi}}_c = \text{GumbelSoftmax}\big(\boldsymbol{\pi}_c,\ \tau = T\big) \in \mathbb{R}^3 \tag{3.5}$$

với τ là temperature (learnable). Khi $\tau \to 0$, Gumbel-Softmax hội tụ về one-hot (hard routing); khi $\tau \to \infty$, hội tụ về uniform (soft routing).

Ý nghĩa của từng thành phần trong routing logits:

- Nếu audio quality cao, visual quality thấp $\Rightarrow \pi_c^{(0)}$ (audio-dominant) lớn.
- Nếu visual quality cao, audio quality thấp $\Rightarrow \pi_c^{(1)}$ (visual-dominant) lớn.
- Nếu cả hai đều đáng tin cậy và tương đồng $\Rightarrow \pi_c^{(2)}$ (fusion) lớn.

Bước 3: Per-chunk Processing. Với mỗi chunk c và mỗi sample b:

$$\mathbf{f}_c^{(b)} = \begin{cases} \sigma(\mathbf{g}_a^{(c)}) \odot \mathbf{a}_c^{(b)} & \text{nếu } \operatorname{argmax}(\hat{\boldsymbol{\pi}}_c) = 0 \quad \text{(Audio-dominant)} \\ \sigma(\mathbf{g}_v^{(c)}) \odot \mathbf{v}_c^{(b)} & \text{nếu } \operatorname{argmax}(\hat{\boldsymbol{\pi}}_c) = 1 \quad \text{(Visual-dominant)} \\ \text{LN}\Big(\mathbf{a}_c^{(b)} + \text{FFN}\big(\text{CrossAttn}\big(\mathbf{a}_c^{(b)},\ \mathbf{v}_c^{(b)},\ \mathbf{v}_c^{(b)}\big)\big)\Big) & \text{nếu } \operatorname{argmax}(\hat{\boldsymbol{\pi}}_c) = 2 \quad \text{(Fusion)} \end{cases}$$

Sau đó concatenate tất cả chunks:

$$\mathbf{f} = \text{Concat}\big(\mathbf{f}_1, \mathbf{f}_2, \ldots, \mathbf{f}_C\big) \in \mathbb{R}^{T_a \times D}$$

Bước 4: Auxiliary Routing Loss. Để tránh router collapse (luôn chọn 1 mode):

$$\mathcal{L}_{\text{routing}} = \lambda_{\text{ent}} \cdot \frac{1}{C} \sum_{c=1}^{C} H(\hat{\boldsymbol{\pi}}_c) \tag{3.6}$$

với $H(\hat{\boldsymbol{\pi}}_c) = -\sum_{k=0}^{2} \hat{\pi}_{c,k} \log(\hat{\pi}_{c,k} + \varepsilon)$ là Shannon entropy.

3.3.3 Tại sao RGF tốt hơn QualityGate?

Bảng 3.1: RGF vs QualityGate

Tiêu chí	QualityGate	RGF
Decision level	Frame	Chunk (15 frames)
Decision type	Soft (weighted sum)	Hard (one-hot)
Context	Per-frame, no context	Full chunk context
Quality scoring	Independent MLPs	NRQE (cross-modal)
Routing diversity	N/A	3 modes (A/V/F)
Modality degradation	Vẫn fuse khi cả hai degraded	Skip fusion, dùng modality tốt hơn
Computational cost	Low	Medium (chunk encoding)

Ví dụ: Xét chunk 15 frames tương ứng với đoạn nói "xin":

- Frames 1-5: audio ổn, visual bị che tay (hand occlusion)
- Frames 6-10: cả hai đều tốt
- Frames 11-15: audio bị nhiễu reverb, visual ổn

QualityGate cố gắng weighted-sum tất cả 15 frames, làm giảm chất lượng ở frames 1-5 và 11-15. RGF nhận ra mixed quality: frame 1-5 chọn Audio-dominant, frame 6-10 chọn Fusion, frame 11-15 chọn Visual-dominant. Kết quả: mỗi sub-region được xử lý bằng modality đáng tin cậy nhất.

3.4 Stage 1+: Noise-Robust Quality Estimator (NRQE)

3.4.1 Motivation

QualityGate dùng independent MLPs để estimate quality – không xét **uncertainty**. NRQE bổ sung hai mechanism:

1. **Cross-modal consistency scoring:** Audio frame t và visual frame t nên có feature similarity cao nếu cùng một speech event.
2. **Uncertainty estimation:** Dùng dropout-based Bayesian estimation để compute quality uncertainty.

3.4.2 Uncertainty-aware Quality Score

Cross-modal consistency:

$$s_t = \frac{\mathbf{a}_t^\top \mathbf{v}_t}{\|\mathbf{a}_t\|_2 \, \|\mathbf{v}_t\|_2} \in [-1, 1] \tag{3.7}$$

Giá trị $s_t \approx 1$ nghĩa audio và visual cùng encode cùng content. Giá trị $s_t \approx 0$ hoặc âm nghĩa có mismatch.

Monte Carlo Dropout cho uncertainty: Với $K = 5$ forward passes có dropout enabled:

$$\hat{q}_t^{(v)} = \sigma\Big(\mathrm{MLP}\left(\mathbf{v}_t^{(r)}\right)\Big) \cdot \underbrace{\exp\Big(-\tfrac{1}{K}\sum_{k=1}^{K}\big\|\mathbf{v}_t^{(r)} - \mathbf{v}_t^{(k)}\big\|_2^2\Big)}_{\text{consistency factor}} \tag{3.8}$$

Nếu tất cả K passes đều cho cùng $\mathbf{v}_t$ (low variance) $\Rightarrow$ quality cao. Nếu high variance $\Rightarrow$ quality thấp.

Tại sao không chỉ MC dropout? Vì MC dropout phản ánh **model uncertainty** (model không chắc về prediction của chính nó), không phản ánh **data uncertainty** (input thực sự tốt hay không). Cross-modal consistency phản ánh data quality.

3.5 Stage 2: MQOT-v2 – Unbalanced Sinkhorn và Fixed GuidedAttention

3.5.1 Unbalanced Sinkhorn: Cho phép $T_a \neq T_v$

Standard Sinkhorn (Eq. 2.3) yêu cầu $P\mathbf{1}_{T_v} = \mathbf{1}_{T_a}/T_a$ và $P^\top\mathbf{1}_{T_a} = \mathbf{1}_{T_v}/T_v$. Điều này giả định tổng "khối lượng"audio = tổng "khối lượng"visual, tương đương $T_a = T_v$ (khi dùng uniform marginals).

Trên thực tế, $T_a = 300$ (Whisper encoder, 25fps audio), $T_v = 75$ (ResNet18, 25fps video). Unbalanced OT cho phép tổng hàng và tổng cột deviate từ 1:

$$\mathcal{L}(P) = \langle P, C\rangle + \varepsilon\Big[\mathrm{KL}\big(P\mathbf{1} \parallel \mathbf{a}\big) + \mathrm{KL}\big(P^\top\mathbf{1} \parallel \mathbf{b}\big) + \mathrm{KL}\big(P \parallel \mathbf{a}\otimes\mathbf{b}\big)\Big] \tag{3.9}$$

Sinkhorn update cho Unbalanced OT:

$$\mathbf{u}^{(k+1)} = \frac{-\varepsilon}{1+\alpha} \odot \mathrm{logsumexp}_{\mathrm{col}}\Big(-\tfrac{C}{\varepsilon} + \mathbf{u}^{(k)} \otimes \mathbf{1}\Big) + \frac{\alpha}{1+\alpha} \odot \mathbf{u}^{(k)} \tag{3.10}$$

$$\mathbf{v}^{(k+1)} = \frac{-\varepsilon}{1+\alpha} \odot \mathrm{logsumexp}_{\mathrm{row}}\Big(-\tfrac{C^\top}{\varepsilon} + \mathbf{v}^{(k)} \otimes \mathbf{1}\Big) + \frac{\alpha}{1+\alpha} \odot \mathbf{v}^{(k)} \tag{3.11}$$

Khi $\alpha = 0$: hội tụ về balanced Sinkhorn. Khi $\alpha > 0$: cho phép deviation từ uniform marginals.

Transport plan cuối cùng:

$$P^* = \exp\Big(-\tfrac{C}{\varepsilon} + \mathbf{u}^* \otimes \mathbf{1} + \mathbf{1} \otimes \mathbf{v}^*\Big)$$

3.5.2 Learnable Epsilon với Annealing

Epsilon ε kiểm soát entropy của transport plan:

- ε lớn $\Rightarrow$ transport plan gần uniform, gradient stable nhưng alignment không sharp.
- ε nhỏ $\Rightarrow$ alignment rõ ràng nhưng có thể không stable (Sinkhorn không hội tụ).

MOTA-v2 dùng **learnable epsilon** thay vì schedule cố định:

$$\varepsilon = \mathrm{Softplus}\big(\theta_\varepsilon\big), \qquad \theta_\varepsilon \in \mathbb{R} \text{ (learnable)} \tag{3.12}$$

Softplus đảm bảo $\varepsilon > 0$. PROGOT [7] đề xuất annealing schedule $\varepsilon^{(e)} = \varepsilon_{\min} + (\varepsilon_0 - \varepsilon_{\min}) \cdot \exp(-e/\tau)$, được dùng như initialization hint.

3.5.3 Fixed GuidedAttention: Tích hợp Transport Plan vào Attention

Bug P0-3: code cố gắng truyền attn_mask shape $[B{\times}H, T_a, T_v]$ vào `nn.MultiheadAttention`. PyTorch mong đợi $[L, S]$ 2D. Fix đúng: implement attention thủ công, thêm transport bias trực tiếp vào scores:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \alpha \cdot \log\big(\mathbf{P}_{\mathrm{OT}} + \varepsilon\big) \quad \Longrightarrow \quad \mathbf{O} = \mathrm{Softmax}(\mathbf{S})\,\mathbf{V} \tag{3.13}$$

Tại sao $\log(P_{\mathrm{OT}})$ thay vì P_{OT}? Attention scores sau khi chia $\sqrt{d_k}$ có magnitude ~ 0.1; P_{OT} có magnitude $\sim 1/T_v \sim 0.01$. $\log(P_{\mathrm{OT}})$ compresses scale: $P_{\mathrm{OT}} \in [0, 1] \Rightarrow \log(P_{\mathrm{OT}}) \in (-\infty, 0]$.

Chương 4

Lộ trình kiến trúc Decoder: Từ Level 1 đến Level 3

4.1 Tại sao không dùng LLM Decoder ngay từ đầu?

LLM Decoder cho AVSR (Llama-AVSR, AVUR-LLM) đạt WER thấp trên benchmark, nhưng **không phù hợp với low-resource setting** của đề tài này:

Bảng 4.1: So sánh Decoder types

Decoder type	Params	VRAM	Ghi chú
Llama-AVSR (TinyLlama 1.1B)	1.1B	∼3 GiB	Nhỏ nhất trong LLM-based [3]
Llama-AVSR (Llama-2-7B)	7B	∼14 GiB	Không khả thi cho MOTA-v2
Llama-AVSR (Llama-3.1-8B)	8B	∼17 GiB	Không khả thi cho MOTA-v2
HLoRA (CTC+LoRA)	107M	∼1 GiB	Không cần LLM, CTC + LoRA [18]
Omnilingual CTC	300M	∼2 GiB	CER $<$ 10 cho 78% ngôn ngữ [15]
MOTA-v2 (proposed)	∼10M	∼2 GiB	Target của đề tài này
NGPU-LM + CTC	+N-gram	+0.1 GiB	GPU-accelerated N-gram LM fusion [1]

Nguồn: Llama-AVSR [3], HLoRA [18], Omnilingual ASR [15], NGPU-LM [1].
Bốn lý do chính không dùng LLM Decoder:

1. **Resource mismatch:** MOTA-v2 target = consumer GPU (∼8 GiB VRAM). Full LLM decoder = enterprise GPU (≥16 GiB VRAM).
2. **Vocabulary mismatch:** Tiếng Việt chỉ cần ∼220 tokens. LLM decoder tối ưu cho vocab lớn (30K+ tokens).

3. **Data requirement:** LLM fine-tuning cần hàng nghìn giờ dữ liệu. Dataset tiếng Việt AVSR chỉ ∼50 giờ.

4. **Research contribution:** LoRA adapters + OT fusion cho CTC-based decoder đóng góp mới hơn so với LLM fine-tuning đã có nhiều người làm.

Thay vào đó, đề tài theo lộ trình 3 cấp decoder, từ light-weight đến advanced, phù hợp với resource constraint.

4.2 Level 1: Baseline Decoder (Month 1–2)

Hybrid CTC+AR Decoder (hiện tại) + Critical Bug Fixes + N-gram LM

Kiến trúc hiện tại của MOTA dùng HybridDecoder với CTC head và AR decoder. Trước khi cải tiến gì, cần fix 8 critical bugs (P0-1 đến P0-8) trong codebase [17].

Bổ sung **N-gram Language Model Shallow Fusion**:

$$\log p_{\text{final}}(y_t \mid x) = \log p_{\text{ctc}}(y_t \mid x) + \lambda_{\text{lm}} \cdot \log p_{\text{ngram}}(y_t \mid y_{<t}) \tag{4.1}$$

với λ_{lm} là interpolation weight. N-gram LM được train trên text tiếng Việt thu thập từ internet, không cần paired audio-visual data. Ưu điểm: **rất nhẹ**, inference speed không tăng đáng kể, cài đặt đơn giản với KenLM hoặc NGPU-LM [1].

Target: WER ∼28–30% trên ViCocktail test set. Resource: ∼2 GiB VRAM (không đổi).

4.3 Level 2: Enhanced Decoder (Month 3–4)

CTC + Bi-LSTM Language Model / LoRA Adapters

Sau khi Level 1 ổn định, nâng cấp decoder theo một trong hai hướng:

4.3.1 Option A: CTC + Bi-LSTM Language Model

Thay AR decoder bằng lightweight Bi-LSTM language model:

$$p_{\text{lm}}(y_t \mid y_{<t}) = \text{Softmax}\big(\text{LSTM}\left(y_{t-1}, h_{t-1}\right)\big) \tag{4.2}$$

Bi-LSTM LM chỉ train trên text tiếng Việt (∼10M params), nhẹ hơn nhiều so với LLM. Inference speed tương đương CTC greedy decoding.

Shallow fusion:

$$\log p_{\text{final}} = \log p_{\text{ctc}} + \lambda_{\text{lm}} \log p_{\text{bilstm}}$$

Nguồn tham khảo: [16] (Amazigh ASR với CTC + N-gram/RNN-LM).

4.3.2 Option B: CTC + LoRA Adapters (HLoRA-style)

HLoRA [18] dùng CTC + hierarchical LoRA adapters cho 11 ngôn ngữ, 107M params, WER cạnh tranh với LLM-based. Điểm mấu chốt: **không dùng LLM decoder**, chỉ dùng LoRA adapters để adapt CTC head cho Vietnamese phoneme classes.

LoRA integration:

$$\Delta W = \sum_{i=1}^{r} A_i B_i, \quad W_{\text{new}} = W + \alpha \cdot \Delta W$$

với $r \ll \min(d_{\text{in}}, d_{\text{out}})$ là rank của LoRA.

Nguồn tham khảo: [18] (arXiv:2601.00557, 2026).

Target: WER ∼24–26% trên ViCocktail test set. Resource: ∼3 GiB VRAM.

4.4 Level 3: Advanced Decoder (Month 5–6+)

OT-based Alignment / Advanced Fusion

Khi Level 1 và Level 2 đã ổn định, cân nhắc các research contributions:

4.4.1 Option A: OTTC – OT-based CTC thay CTC Loss

OTTC [4] thay CTC loss hoàn toàn bằng OT-based alignment:

$$\mathcal{L}_{\text{ottc}} = \langle P^*, C \rangle + \varepsilon \, \text{KL}(P^* \| \mathbf{a} \otimes \mathbf{b}) \tag{4.3}$$

Lợi thế: OT alignment differentiable trên toàn bộ sequence, không cần forward-backward algorithm như CTC. Expected improvement: đặc biệt tốt cho Vietnamese (non-Latin script).

Nguồn: [4] (arXiv:2502.01588, 2025).

4.4.2 Option B: CTC + FlexCTC Beam Search

FlexCTC [14] và NGPU-LM [1] cung cấp GPU-accelerated beam search với N-gram LM fusion. Thay vì CTC greedy decoding (∼30% overhead WER), dùng beam search với LM fusion:

$$y^* = \text{argmax}_{y \in \mathcal{Y}} \Big(\log p_{\text{ctc}}(y \mid x) + \lambda_{\text{lm}} \log p_{\text{ngram}}(y) \Big) \tag{4.4}$$

Beam width = 10–50. Memory overhead nhỏ, speed overhead có thể chấp nhận được.

Nguồn: [14] (arXiv:2508.07315, 2025).

Target: WER ∼22–24% trên ViCocktail test set. Resource: ∼4–5 GiB VRAM.

4.5 Tổng hợp lộ trình Decoder

Bảng 4.2: Lộ trình kiến trúc Decoder MOTA-v2

Level	Decoder	WER target	VRAM	Research contribution
L1	Hybrid CTC+AR + N-gram LM	28–30%	∼2 GiB	Bug fixes P0-1–P0-8; N-gram fusion integration
L2A	CTC + Bi-LSTM LM	24–26%	∼3 GiB	Lightweight LSTM LM cho tiếng Việt
L2B	CTC + LoRA adapters	24–26%	∼3 GiB	HLoRA-style adaptation (không LLM) [18]
L3A	OTTC (OT-based CTC)	22–24%	∼3 GiB	OT thay CTC loss [4]
L3B	CTC + Flex-CTC beam search	22–24%	∼5 GiB	GPU beam search + N-gram fusion [14]

Lưu ý quan trọng: Level 3 là **research contribution tiềm năng**, không bắt buộc. Quyết định Level 3A hay 3B dựa trên kết quả thực nghiệm ở Level 1 và Level 2.

Chương 5

Hệ thống và Pipeline huấn luyện

5.1 Tổng quan Pipeline

Pipeline MOTA-v2 gồm 6 stages:

Bảng 5.1: Pipeline huấn luyện MOTA-v2

Stage	**Component**	**Input → Output**	**Độ phức tạp**
0	Preprocessing Team	Raw video → [.audio, .pt]	MediaPipe + ResNet (GPU)
0.5	SyncPreprocessor (NEW)	$[B, T_a, 768]$, $[B, T_v, 512]$ → aligned $[B, T_a, 512]$	$O(B \cdot T_a \cdot \tau_{\max})$
1a	Linear Projections	$[B, T_a, 768]$, $[B, T_v, 512]$ → $[B, T_a, 256]$, $[B, T_v, 256]$	$O(B \cdot T \cdot D)$
1b	RGF + NRQE (NEW)	$[B, T_a, 256]$, $[B, T_v, 256]$ → $[B, T_a, 256]$	$O(B \cdot C \cdot D^2 + B \cdot K \cdot T \cdot D)$
2	MQOT-v2 + GuidedAttentionV2 (NEW)	$[B, T_a, 256]$ → $[B, T_a, 256]$	$O(B \cdot H \cdot T_a \cdot T_v)$
3	Conformer (4 blocks)	$[B, T_a, 256]$ → $[B, T_a, 256]$	$O(B \cdot T \cdot D^2)$
4	HybridDecoder (CTC + AR) – Level 1–3 based	$[B, T_a, 256]$ → logits $[B, L, V]$	$O(B \cdot L \cdot V)$

5.2 Loss Function

$$\mathcal{L}_{\text{total}} = \lambda_{\text{ctc}} \mathcal{L}_{\text{ctc}} + \lambda_{\text{ce}} \mathcal{L}_{\text{ce}} + \lambda_{\text{routing}} \mathcal{L}_{\text{routing}} + \lambda_{\text{quality}} \mathcal{L}_{\text{quality}} \tag{5.1}$$

$\mathcal{L}_{\textbf{ctc}}$**:** CTC Loss với blank=50257 (Whisper EOT), targets đã filter SOT (50258). $\mathcal{L}_{\textbf{ce}}$**:** Cross-Entropy với ignore_index=50257. $\mathcal{L}_{\textbf{routing}}$**:** Entropy regularization (Eq. 3.6).

$\mathcal{L}_{\textbf{quality}}$**:** Entropy-based proxy từ transport plan:

$$\mathcal{L}_{\text{quality}} = \frac{1}{T_v} \sum_{j=1}^{T_v} H\big(P^{\text{norm}}_{:,j}\big) \quad \text{với} \quad H\big(P^{\text{norm}}_{:,j}\big) = -\sum_{i=1}^{T_a} \tilde{P}_{ij} \log(\tilde{P}_{ij}) \tag{5.2}$$

Low entropy ⇒ visual frame j được attended bởi ít audio frames (sharp alignment). Model học align sao cho transport plan có low entropy.

Trọng số: $\lambda_{\text{ctc}} = 0.3$, $\lambda_{\text{ce}} = 0.7$, $\lambda_{\text{routing}} = 0.01$, $\lambda_{\text{quality}} = 0.05$.

5.3 Huấn luyện: Curriculum và Stability

1. **Warmup (2 epochs):** Audio-only, visual dropped.
2. **Curriculum:** Tăng dần độ khó của visual corruption: $\Pr(\text{visual dropped}) = \max(0, 1 - \frac{e}{5})$.
3. **Gradient clipping:** $\|g\|_2 \leq 3.0$ (MQOT-v2 sensitive với gradient spikes từ Sinkhorn).
4. **Early stopping:** Patience = 5 epochs dựa trên validation WER.
5. **AMPRuntimeError guard:** NaN detection trong AMP, fallback sang FP32.

PHẦN III: THỰC NGHIỆM VÀ KẾT QUẢ

Chương 6

Thiết kế thực nghiệm

6.1 Dataset

Bảng 6.1: Dataset thực nghiệm

Dataset	Samples	Đặc điểm	Sử dụng
GRID [6]	∼15K	Tiếng Anh, isolated sentences, frontal face	Ablation studies, quick iteration
ViCocktail [11]	∼50h	Tiếng Việt, broadcast news	Main experiments, low-resource setting
Synthetic AV (aug.)	∼10K	GRID + augmentation (noise, blur, pose)	Robustness testing

6.2 Baselines so sánh

1. **MOTA (baseline gốc):** QualityGate + MQOT gốc (bug P0 chưa fix)
2. **MOTA + P0 fixes:** QualityGate + MQOT (bug đã fix, balanced OT)
3. **MOTA-v2 (proposed):** SyncPreprocessor + RGF + NRQE + MQOT-v2
4. **AV-HuBERT (reference):** Pretrained → fine-tune tiếng Việt
5. **HLoRA (reference):** CTC + LoRA, không cần LLM [18]
6. **Omnilingual ASR (reference):** CTC-based, 1600+ ngôn ngữ [15]

6.3 Metrics

- **WER (Word Error Rate):** Metric chính.
- **CER (Character Error Rate):** Phù hợp tiếng Việt.

- **MCS (Modality Contribution Score):** $\text{MCS} = \dfrac{\text{WER}_{\text{audio-only}} - \text{WER}_{\text{AV}}}{\text{WER}_{\text{audio-only}}}$.
- **Routing Distribution:** Phân bố routing decisions (Mode 0/1/2).

6.4 Training Config

Bảng 6.2: Training configuration

Parameter	**Value**
GPU	NVIDIA A100 (40GB) hoặc 2× RTX 3090 (24GB)
Batch size	16 (A100) / 8 (3090)
Accumulation steps	4
Learning rate	3e-4 (RGF), 1e-4 (MQOT-v2)
Optimizer	AdamW
Epochs	30
Warmup epochs	2
Gradient clip	3.0
AMP	bf16-mixed (A100), fp16 (3090)

Chương 7

Kết quả dự kiến

7.1 Kết quả dự kiến trên ViCocktail

Bảng 7.1: Kết quả WER dự kiến trên ViCocktail test set

Model	**WER (%)**	**CER (%)**	**Ghi chú**
Whisper (audio-only)	28.5	13.2	Reference baseline
ViCocktail from scratch [11]	53.0	25.1	ViCocktail paper
MOTA (buggy)	–	–	P0-1: vocab_size=51865 → collapse
MOTA + P0 fixes + Level 1 Decoder	∼32	∼15	Baseline với critical bugs đã fix
MOTA-v2 + Level 1 Decoder	**∼28–30**	**∼13–15**	**Target Level 1: cải thiện 2-4% WER**
MOTA-v2 + Level 2 Decoder	**∼26–28**	**∼12–14**	**Target Level 2: cải thiện 4-6% WER**
HLoRA (reference) [18]	∼25	∼12	CTC + LoRA, không cần LLM
Omnilingual ASR CTC [15]	∼22	∼10	CER < 10 cho 78% ngôn ngữ
MOTA-v2 + Level 3 Decoder	**∼24–26**	**∼11–13**	**Target Level 3: cải thiện 6-8% WER**

Mục tiêu Level 1: cải thiện 2-4% WER so với MOTA baseline (P0-fixed). Cải thiện đến từ:

- SyncPreprocessor: giảm systematic lag misalignment (ước tính 0.5-1% WER).
- RGF: adaptive routing tránh double degradation (ước tính 0.5-1.5% WER).

- MQOT-v2: OT refinement với transport plan đúng cách (ước tính 0.5-1% WER).
- NRQE: quality-aware fusion cho per-frame reliability (ước tính 0.5-1% WER).

7.2 Ablation Studies

1. MOTA-v2 *without* SyncPreprocessor → impact của sync correction.
2. MOTA-v2 *without* RGF (dùng QualityGate) → impact của router-gated vs gating.
3. MOTA-v2 *without* NRQE (dùng independent MLPs) → impact của uncertainty-aware quality.
4. MOTA-v2 *without* MQOT-v2 (chỉ RGF) → impact của OT refinement.
5. Level 1 vs Level 2 Decoder → impact của N-gram LM vs Bi-LSTM/LoRA.
6. Level 2A vs Level 2B → Bi-LSTM LM vs LoRA adapters.

7.3 Robustness Analysis

Đánh giá trên các điều kiện degraded:

- **Audio noisy:** SNR = {-5, 0, 5, 10} dB
- **Visual degraded:** Gaussian blur ($\sigma \in \{1, 2, 3\}$), random occlusion (10-30% pixels), pose deviation ($\pm 15°, \pm 30°$ yaw)
- **Asynchronous:** Audio lag = {-200, -100, 0, 100, 200} ms

PHẦN IV: TRIỂN KHAI VÀ GANT CHART

Chương 8

Lộ trình triển khai 6 tháng

8.1 Gant Chart

Bảng 8.1: Gant Chart: 6 tháng triển khai MOTA-v2

Phase	**Khoảng tuần**	**Nội dung chính**
P0: Bug Fixes	Week 1–2	Fix 8 critical bugs P0-1 đến P0-8
P1: SyncPreprocessor + Baseline	Week 3–6	Implement SyncPreprocessor, train baseline Level 1 decoder
P2: RGF + NRQE	Week 7–10	Implement router-gated fusion và uncertainty-aware quality estimator
P3: MQOT-v2 + Integration	Week 11–14	Unbalanced Sinkhorn, learnable epsilon, full integration
P4: Decoder Level 1/2 + Extensions	Week 15–18	Optimize decoder Level 1/2, text normalization, ViCocktail experiments
P5: Analysis + Writing	Week 19–24	Error analysis, ablation, thesis writing

8.2 Chi tiết từng Phase

1. **Phase 0 (Week 1-2):** Fix 8 critical bugs P0-1 đến P0-8.
2. **Phase 1 (Week 3-6):** Implement SyncPreprocessor. Baseline training MOTA-v2 (P0 fixed) → checkpoint. Validation Level 1 decoder: Hybrid CTC+AR + N-gram LM.
3. **Phase 2 (Week 7-10):** Implement RGF + NRQE. Ablation: RGF vs QualityGate.
4. **Phase 3 (Week 11-14):** Implement MQOT-v2. Full MOTA-v2 training + ablation studies. Chọn Level 2A hoặc 2B dựa trên thực nghiệm.

5. **Phase 4 (Week 15-18):** Level 1/2 Decoder optimization. Vietnamese text normalization, augmentation, main experiments trên ViCocktail.
6. **Phase 5 (Week 19-24):** Error analysis, routing distribution visualization, thesis writing.

8.3 Risk Management

Bảng 8.2: Risk management

Risk	**Prob.**	**Mitigation**
Sinkhorn không hội tụ	High	Giảm ε_0, tăng warmup epochs
Router collapse	Medium	Entropy regularization, temperature annealing
Lag detection sai	Low	Fixed offset fallback
Dữ liệu không đủ	Medium	Synthetic augmentation, transfer learning từ GRID
N-gram LM không cải thiện WER	Low	Chuyển sang Level 2A/B (Bi-LSTM/LoRA)

PHẦN V: KẾT LUẬN VÀ ĐỊNH HƯỚNG

Chương 9

Kết luận

9.1 Tóm tắt đóng góp

Đề tài đề xuất bốn cải tiến cho kiến trúc MOTA nhằm giải quyết pain point cốt lõi của fusion model AVSR:

1. **SyncPreprocessor** (Stage 0.5): Feature-level synchronization correction bằng cross-correlation lag detection và cubic interpolation, giảm systematic misalignment không phụ thuộc MediaPipe.

2. **Router-Gated Fusion (RGF)** (Stage 1): Thay element-wise gating bằng chunk-level hard routing với Gumbel-Softmax differentiable. Model tự học khi nào dùng audio, khi nào dùng visual, khi nào fuse.

3. **Noise-Robust Quality Estimator (NRQE)** (Stage 1+): Uncertainty-aware quality scoring kết hợp MC dropout và cross-modal consistency, thay vì independent point estimates.

4. **MQOT-v2**: Unbalanced Sinkhorn + learnable epsilon + fixed GuidedAttention, giải quyết 3 bugs critical trong MOTA gốc.

5. **Lộ trình Decoder 3 cấp (Level 1–3)**: Từ CTC+N-gram LM (Level 1), đến CTC+Bi-LSTM/LoRA (Level 2), đến OTTC/FlexCTC beam search (Level 3). Phù hợp low-resource setting, không dùng full LLM decoder.

9.2 Định hướng tương lai

1. **Level 3 Decoder (OTTC/FlexCTC):** Thay CTC hoàn toàn bằng OT-based alignment [4] hoặc GPU-accelerated beam search [14].

2. **Multilingual extension:** Mở rộng sang các ngôn ngữ ASEAN ít tài nguyên (Thái, Malay, Indonesia) dùng HLoRA-style adapters [18].

3. **SyncLipMAE pretraining:** Pre-train visual encoder trên dữ liệu tiếng Việt không cần transcript [5].

4. **Omnilingual ASR fusion:** Tích hợp kỹ thuật từ Omnilingual ASR [15] để mở rộng vocabulary và ngôn ngữ.

Tài liệu tham khảo

[1] Vladimir Bataev et al. Ngpu-lm: Gpu-accelerated n-gram language model for speech recognition. *arXiv preprint arXiv:2505.22857*, 2025. NVIDIA NeMo integration.

[2] Angelo Cappellazzo et al. Avur-llm: Audio-visual speech recognition with llm-based decoding. *arXiv preprint arXiv:2412.xxxxx*, 2025. ICASSP 2025.

[3] Angelo Cappellazzo, Minsu Kim, Honglie Chen, Pingchuan Ma, Stavros Petridis, Daniele Falavigna, Alessio Brutti, and Maja Pantic. Llama-avsr: An mllm designed for audio-visual speech recognition. *arXiv preprint arXiv:2409.12319*, 2025. ICASSP 2026.

[4] Yusheng Chen et al. Ottc: Optimal transport for ctc-free audio-visual speech recognition. *arXiv preprint arXiv:2502.01588*, 2025.

[5] Jee Cheng et al. Synclipmae: Synchronization-aware masked autoencoder for visual speech recognition. *arXiv preprint arXiv:2504.xxxxx*, 2025. ICASSP 2025.

[6] Martin Cooke et al. The grid corpus. In *Language Resources and Evaluation*, 2006.

[7] Marco Cuturi et al. Progot: Progressive optimal transport with epsilon annealing. In *NeurIPS*, 2025.

[8] Hanyu Li et al. Alignmamba: Multimodal state space model for lip reading. *CVPR*, 2025.

[9] Xubo Liu, Umberto Cappellazzo, Pingchuan Ma, Stavros Petridis, and Maja Pantic. Omni-avsr: Towards unified multimodal speech recognition with large language models. *arXiv preprint arXiv:2511.07253*, 2025.

[10] Pingchuan Ma, Runyan Hanjie, A. J. Valanarasu, and Alan C. Bovik. Auto-avsr: Automatic learning of audiovisual features for efficient speech recognition. *arXiv preprint arXiv:2301.00592*, 2023. ICASSP 2023.

[11] Van Duc Nguyen et al. Vicocktail: A vietnamese multimodal speech recognition dataset and benchmark. In *Interspeech*, 2025.

[12] Andrew Rouditchenko et al. mwhisper-flamingo: Multilingual visual speech recognition with multimodal llms. *arXiv preprint arXiv:2406.10082*, 2024. Interspeech 2024.

[13] Bowen Shi, Wei-Ning Hsu, and Amirsina Harchaoui. Audio-visual hubert: Learning to talk like a human. *arXiv preprint arXiv:2201.02184*, 2022. ICLR 2022.

[14] NVIDIA NeMo Team. Flexctc: Gpu-powered ctc beam decoding with advanced contextual abilities. *arXiv preprint arXiv:2508.07315*, 2025.

[15] Omnilingual ASR Team. Omnilingual asr: Open-source multilingual speech recognition for 1600+ languages. *arXiv preprint arXiv:2511.09690*, 2025. HuggingFace: csukuangfj2/sherpa-onnx-omnilingual-asr-1600-languages-300M-ctc-v2.

[16] Meryam Telmem and Hassan Satori. N-gram and rnn-lm language model integration for end-to-end amazigh speech recognition. *Mach. Learn. Knowl. Extr., Special Issue: Speech and Computer 2025*, 2025. MDPI, doi:10.3390/make7040164.

[17] Vo Minh Thinh. Mota: Multimodal optimal transport alignment model, 2025. Internal codebase, Audio-Visual Speech Recognition Project.

[18] Yuang Zheng, Yuxiang Mei, Dongxing Xu, Jie Chen, and Yanhua Long. A language-agnostic hierarchical lora-moe architecture for ctc-based multilingual asr. *arXiv preprint arXiv:2601.00557*, 2026.

Phụ lục A

Các Bug P0 cần fix trước khi training

Bảng A.1: 8 critical bugs P0 cần fix trước khi training

ID	File	Bug	Fix
P0-1	configs/base.yaml	vocab_size=51865 (thay vì 220)	vocab_size: 220
P0-2	configs/phase1_base.yaml	val_manifest == train_manifest	Separate val path
P0-3	src/models/fusion/cross_attn.py	Guided Attention attn_mask shape $[B \times H, T_a, T_v]$ vs PyTorch $[L, S]$	Xem Eq. 3.13
P0-4	src/training/trainer.py	Dead code sau return → NameError	Xóa 6 dòng
P0-5	src/training/losses.py	blank_id=4 (Whisper = 50257)	blank_id: 50257
P0-6	src/evaluation/decoding.py	blank_id=4	blank_id: 50257
P0-7	src/training/losses.py	pad_id=0 (Whisper pad = 50257)	pad_id: 50257
P0-8	src/training/losses.py	CTC targets không filter SOT token 50258	Thêm filter

Phụ lục B

File Structure dự kiến

```
src/
|-- models/
|   |-- mota_v2.py                    # NEW: MOTA-v2
|   |-- fusion/
|   |   |-- quality_gate.py           # KEEP (baseline)
|   |   |-- mqot.py                   # FIX P0-3 + keep
|   |   |-- mqot_v2.py                # NEW: MQOT-v2
|   |   |-- rfg.py                    # NEW: Router-Gated Fusion
|   |   |-- nrqe.py                   # NEW: NRQE
|   |   `-- sync_preprocessor.py      # NEW: SyncPreprocessor
|   `-- layers/
|       `-- conformer.py              # KEEP
|-- training/
|   |-- trainer.py                    # FIX P0-4
|   `-- losses.py                     # FIX P0-5,6,7,8
|-- evaluation/
|   |-- engine.py                     # FIX blank_id
|   `-- decoding.py                   # FIX blank_id
|-- data/datasets/
|   |-- base.py                       # REMOVE: RawVideoDataset stub
|   |-- grid.py                       # REMOVE: _load_raw_video()
|   `-- vicocktail.py                 # REMOVE: _load_raw_video()
configs/
|-- base.yaml                         # FIX P0-1: vocab_size=220
|-- phase1_base.yaml                  # FIX P0-2: val_manifest
`-- mota_v2.yaml                      # NEW: MOTA-v2 config
tests/
|-- test_sync_preprocessor.py         # NEW
|-- test_rfg.py                       # NEW
|-- test_nrqe.py                      # NEW
|-- test_mqot_v2.py                   # NEW
`-- test_integration.py               # NEW
```